Enterprise SOC Lab Project Documentation

Project purpose and current milestone

I am building a home security operations center (SOC) lab to practice endpoint monitoring, log analysis, alert investigation, and documentation.

My lab uses Wazuh as the main Security Information and Event Management (SIEM) for collecting and analyzing security information from my multiple systems.

So far, my Ubuntu server runs the Wazuh manager, indexer, and dashboard in Docker. My main Windows computer and Windows virtual machine have active Wazuh agents, and I have verified that events from the VM appear in the dashboard.

1: Purchase and verify the homelab computer

I Purchased a Lenovo M710 mini PC off of eBay with:

- Intel i5
- 16gb Ram
- 256 gb SSD
- Windows 11 originally installed

Before changing the operating system, I verified that it powered on, matched the advertised specifications, and connected to the internet.

Step 2: Install Ubuntu Server

On another computer, I downloaded an Ubuntu Server ISO from the official Ubuntu website and downloaded Rufus.

Using Rufus, I selected the Ubuntu ISO and wrote it onto a USB drive. Then, I inserted the USB drive into the Lenovo and booted from it. This started the Ubuntu installation.

Once complete, I setup my homelabs storage configuration, user account, and network settings. At this point, I had the operating system foundation for my homelab.

Step 3: Organizing the physical network

I reorganized my homelab to contain more professional and useful equipment such as:

- **GeekPi 10” server rack**: Allows me to store all my Homelab network devices into one area
- **GeekPi 12 patch panel**: Allows my devices to cleanly connect into one panel
- **TP-Link 8 port switch, unmanaged**: Allows me direct plug and play when connecting the different ethernet ports/cables to my different devices
- **Short CAT6 patch cables**: allows me to connect my patch panel and network switch together for my devices

The patch panel is passive cable organization. The switch connects devices, the router handles routing and, in my setup, assigns LAN addresses.

Step 4: Enabling remote administration

For a fresh Ubuntu installation, these commands allowed me to install and enable the SSH server:

- sudo apt update
- sudo apt install openssh-server
- sudo systemctl enable --now ssh

Using my main computer I can then easily SSH onto my homelab. Afterwards, I installed Tailscale for a secure remote connection. This helps me avoid needing to expose SSH through port forwarding.

Step 5: Install Docker and Deploy Wazuh

For my chosen deployment method, the order is:

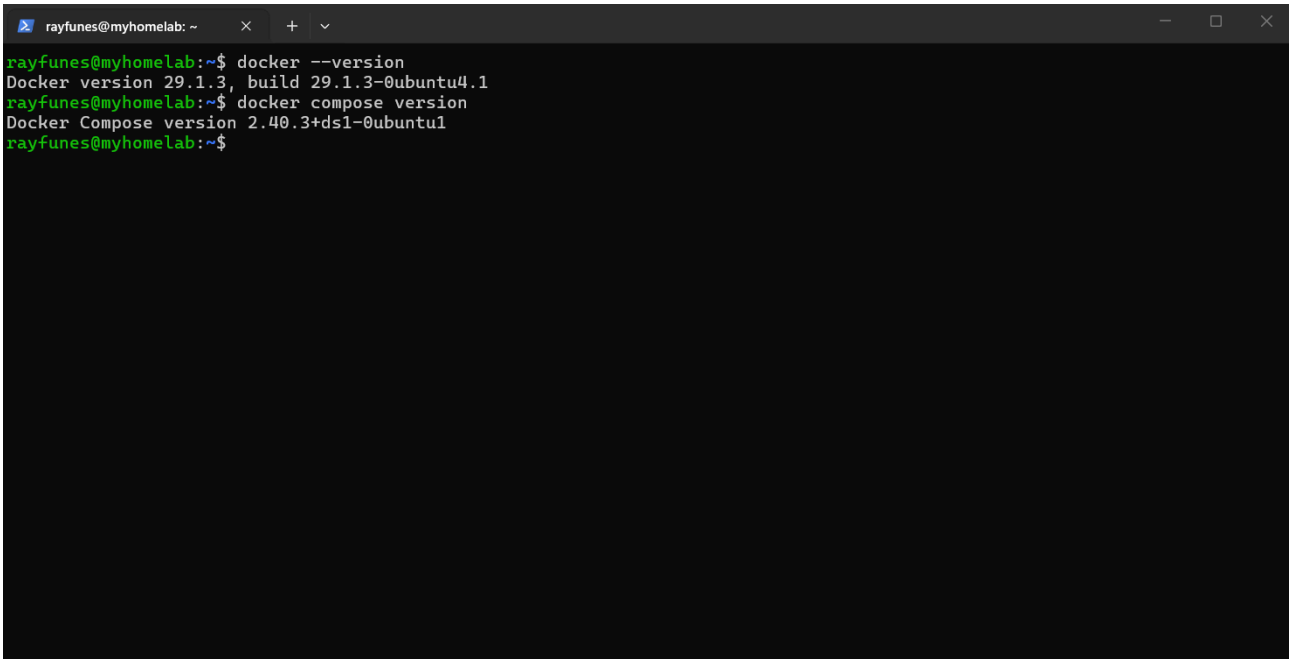
1. Install Ubuntu
2. Install Docker Engine and the Docker Compose Plugin
3. Download the Wazuh Docker deployment configuration
4. Configure and start the Wazuh containers
5. Access the dashboard
6. Install any agents on the computers I want to monitor

I installed Docker Engine and the Compose plugin using the official Docker Ubuntu instructions, located here → https://docs.docker.com/engine/install/ubuntu/

After the installation process, I verified them with these commands:

- docker --version
- docker compose version

Here is that screenshot for even more verification:



Step 6: Understanding where each Wazuh component runs

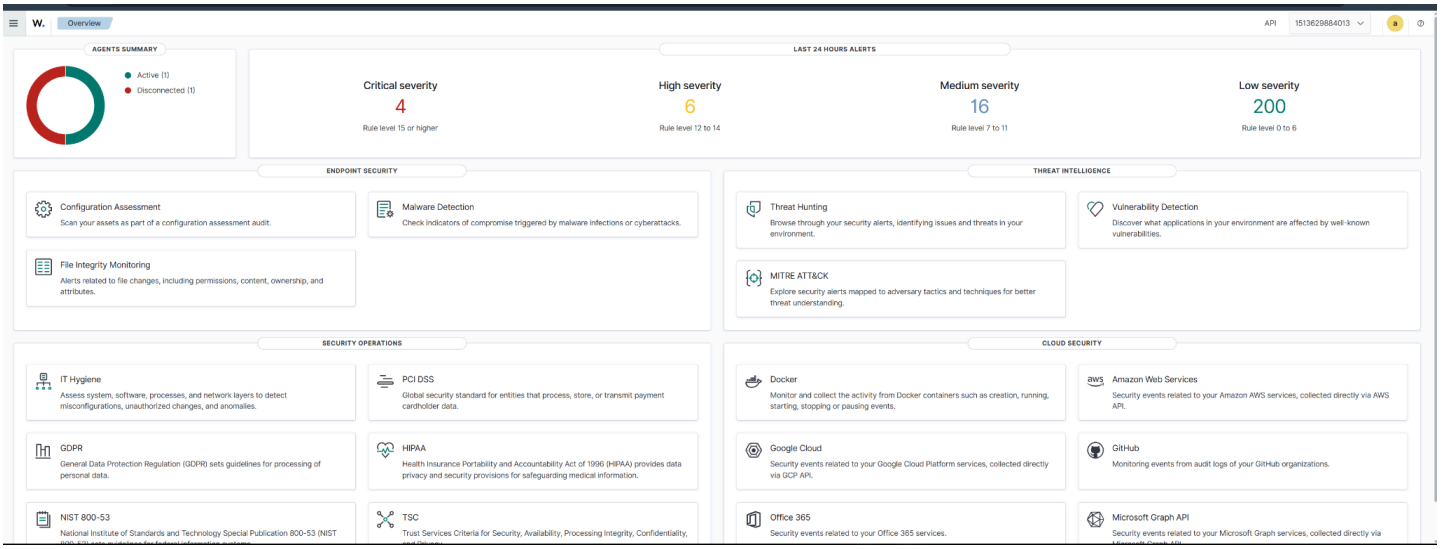
Component	Location	Purpose
Wazuh Agent	My main computer and Windows VM	Collects logs and information about that computer and sends them to the manager.
Wazuh Manager	Ubuntu homelab	Analyzes incoming data using detection rules and generates alerts.
Wazuh Indexer	Ubuntu homelab	Stores and organizes security data so it can be searched.
Wazuh Dashboard	Ubuntu homelab	Gives me a visual browser interface for investigating the data.

The basic flow is that the agent sends information to the manager, the manager analyzes it, and indexed security data becomes available in the dashboard for me to view.

Step 7: Access the dashboard

Example → https://IP_ADDRESS:8443

In my deployment, host port 8443 maps to dashboard container port 5601. The browser then displays the application hosted on Ubuntu. The dashboard also uses the manager API for management information. I'm also able to view the dashboard away from home using Tailscale.



Step 8: Building a Windows test endpoint

On my main Windows PC, I used Oracle VirtualBox to create a fresh VM named Win11-Client-01 and installed Windows 11 from a Windows ISO. This VM is my dedicated endpoint for my controlled lab exercises.

Windows VM configuration:

Setting	Configuration
Virtual CPUS	3
Ram	8 gb
Virtual Disk	80 gb, dynamically allocated VDI
Firmware	EFI
TPM	2.0
Secure Boot	Enabled
Networking	NAT
Observed guest IP	12.3.4.56

Challenges I faced:
During setup, I encountered many black screens. The VM finally reached the installation after switching from 2 to 3 virtual CPUs.

I also bypassed the microsoft login attempt by entering the following command in Command Prompt:

- OOBE\BYPASSNRO

Step 9: Test connectivity from the Virtual Machine

Once inside the Windows VM, I opened PowerShell and ran:

- Test-NetConnection 192.123.4.567 -Port 1515
- Test-NetConnection 192.123.4.567 -Port 1514

TCP 1515 is used for agent enrollment
TCP 1514 is used for agent communication

A successful test meant a TCP connection could be established. This does not install the agent though.

Step 10: Deploy the Wazuh agent
Back in my Wazuh dashboard, I opened Endpoint → Deploy new agent
I then selected Windows and entered:

Field	Value
Manager address	192.123.4.567
Agent name	Win11-Client-01
Group	default

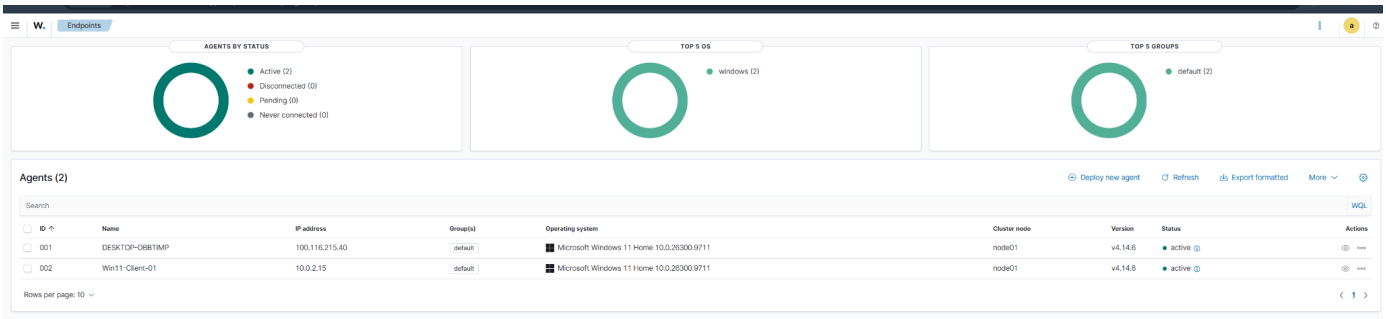
Then I copied the generated installation command and ran it in PowerShell as Administrator inside the VM. This installs the Windows agent using the supplied deployment settings.

Afterwards, I ran:

- Start-Service WazuhSvc
- Get-Service WazuhSvc

Start-Service starts the installed service, while Get-Service reports its local status.

Then I went back to my Wazuh dashboard in order to check that the new agent was enrolled and running:



As you can see above, both agents are showing version 4.14.6.

My main computer was already enrolled. The new VM became the second monitored endpoint within my SIEM environment.

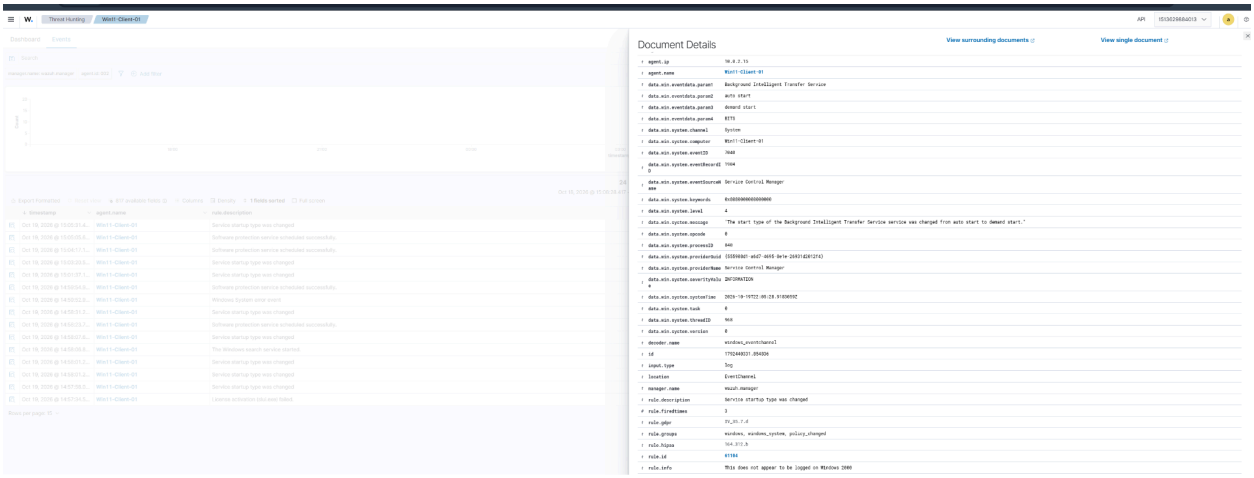
Step 11: Verify and investigate a received event

In Threat Hunting, I selected Win11-Client-01, agent 002, and opened an event

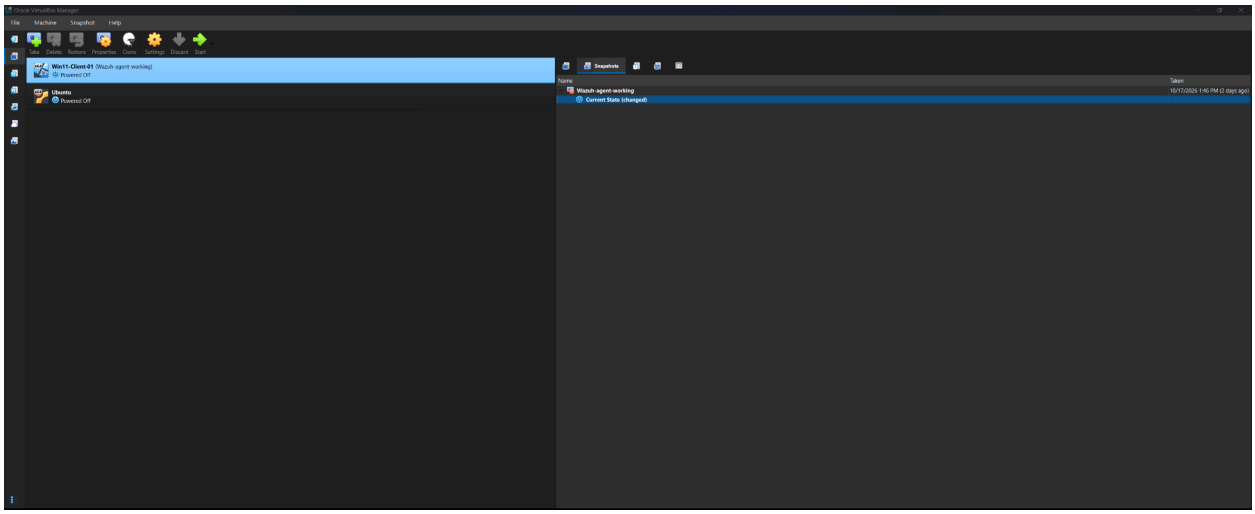
The dashboard showed hundreds of alerts in the selected period. An alert count is not an attack count. One reviewed event contained:

Field	Observed value
Windows event ID	7040
Channel	System
Provider	Service Control Manager
Change	Auto Start
Wazuh rule ID	61104
Rule description	Service startup type was changed
Wazuh rule level	3

Here is that screenshot:



This confirmed that an event from the VM was collected, analyzed, indexed, and displayed in the dashboard. Afterwards, I created a snapshot named Wazuh-agent-working. This provides a working restore point for lab exercise.



Step 12: Installing Sysmon to the Windows VM and Connecting it to Wazuh

I installed Sysmon on my Windows lab VM to collect more detailed information about activity on the endpoint. I then configured the Wazuh agent to collect Sysmon events and verified that they appeared in the Wazuh dashboard.

First I visit Microsoft's official download page and download the zip file located here → https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon

Next I run PowerShell as Administrator and run these commands to get to the extracted folder and install Sysmon

- cd C:\Tools\Sysmon
- dir
- .\Sysmon64.exe -i
- Get-Service Sysmon*

After confirming Sysmon was installed correctly and was running, I then set up the Wazuh agent configuration

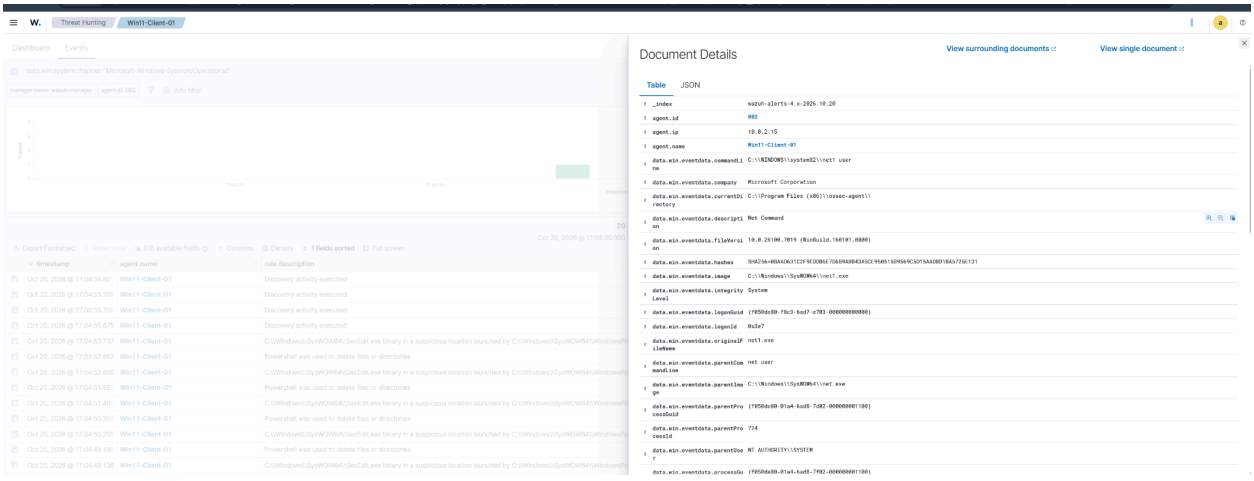
- notepad "C:\Program Files (x86)\ossec-agent\ossec.conf"

After opening the configuration file, I proceeded to add the following block to allow the agent to collect Sysmon events

```
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
```

I then restarted the Wazuh agent to load the configuration change and verified Sysmon events in the Wazuh dashboard by opening Threat Hunting → Events

- Restart-Service WazuhSvc
- Get-Service WazuhSvc



This screenshot shows that a Sysmon process creation event was successfully collected, analyzed, indexed, and displayed all within the Wazuh dashboard.

Step 13: Investigation 01 - Controlled Local Repeated Login Failure Simulation

Investigation 01: Repeated Failed Logins and Account Lockout

Date: October 30, 2026
Endpoint: Win11-Client-01
Wazuh agent ID: 002
Test account: SOC-Test

Objective:
Test whether Wazuh could detect individual failed Windows logins, correlate repeated failures, and report an account lockout

Environment and Preparation:
I used my Windows 11 VirtualBox VM, which has the Wazuh agent installed and connected to my Ubuntu-based Wazuh server. I took a snapshot before testing and created a dedicated local account named SOC-Test to avoid locking out my administrator account.

I checked Windows logon auditing:

- auditpol /get /subcategory:"Logon"

The result showed that both successful and failed logons were being audited.

Next, I created the test account:

- net user SOC-Test * /add

The * prompted me to enter its password without displaying it.

Next, I checked the account lockout policy:

- net accounts

The policy showed a lockout threshold of 10 failed attempts, a lockout duration of 10 minutes, and an observation window of 10 minutes.

Test Procedure:
Inside the Windows VM, I opened Command Prompt and ran:

- runas /user:Win11-Client-01\SOC-Test cmd.exe

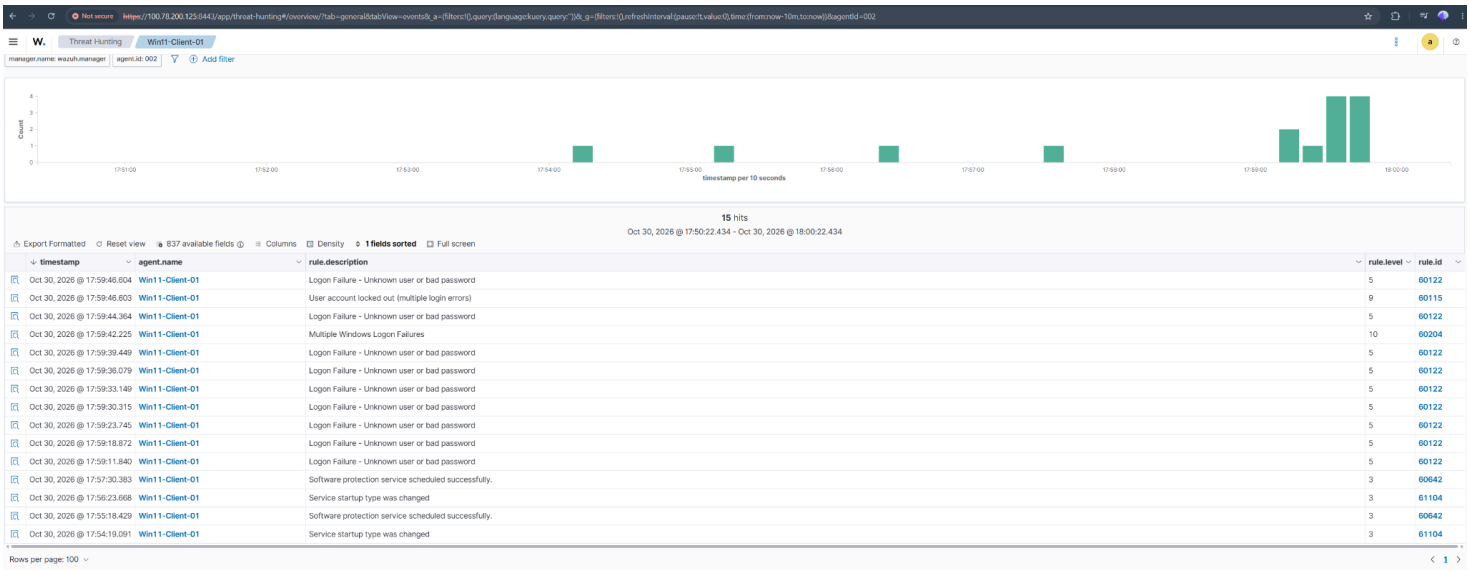
This attempted to launch Command Prompt using the test account. I deliberately entered an incorrect password and received error 1326, indicating that the username or password was incorrect.

I repeated the failed login attempts over a short period, then checked the events in Wazuh under Threat Hunting → Events, filtered to agent 002.

Results:

The dashboard showed the following sequence. Times below are displayed dashboard times.

Observation	Wazuh rule	Level	Time
Individual failed logins	60122	5	Multiple entries before the correlated alert
Multiple Windows logon failures	60204	10	17:59:42.225
User account lockout	60115	9	17:59:46.603



Document Details

View surrounding documents | View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.30
t agent.id	002
t agent.ip	10.0.2.15
t agent.name	Win11-Client-01
t data.win.eventdata.authenticationPackageName	Negotiate
t data.win.eventdata.failureReason	%%2313
t data.win.eventdata.ipAddress	::1
t data.win.eventdata.ipPort	0
t data.win.eventdata.keyLength	0
t data.win.eventdata.logonProcessName	seclogo
t data.win.eventdata.logonType	2
t data.win.eventdata.processId	0x1b6c
t data.win.eventdata.processName	C:\\Windows\\System32\\svchost.exe
t data.win.eventdata.status	0xc000006d
t data.win.eventdata.subStatus	0xc000006a
t data.win.eventdata.subjectDomainName	Win11-Client-01
t data.win.eventdata.subjectLogonId	0x808a7
t data.win.eventdata.subjectUserName	socadmin
t data.win.eventdata.subjectUserSid	S-1-5-21-481501202-2694083569-1166702774-1001
t data.win.eventdata.targetDomainName	Win11-Client-01
t data.win.eventdata.targetUserName	SOC-Test

Document Details

View surrounding documents | View single document

t data.win.system.task	12544
t data.win.system.threadID	8072
t data.win.system.version	0
t decoder.name	windows_eventchannel
t id	1793401182.885876
t input.type	log
t location	EventChannel
t manager.name	wazuh.manager
t previous_output	{"win":{"system":{"providerName":"Microsoft-Windows-Security-Auditing","providerGuid":"{54849625-5478-4994-a5ba-3e3b0328c30d}","eventID":"4625","version":"0","level":"0","task":"12544","opcode":"0","keywords":"0x8010000000000000","systemTime":"2026-10-31T00:59:37.3963378Z","eventRecordID":"46982","processID":"856","threadID":"924","channel":"Security","computer":"Win11-Client-01","severityValue":"AUDIT_FAILURE","message":"\"An account failed to log on.\r\n\r\nSubject:\r\n\tSecurity ID:\t\tS-1-5-21-481501202-2694083569-1166702774-1001\r\n\tAccount Name:\t\tsocadmin\r\n\tAccount Domain:\t\tWin11-Client-01\r\n\tLogon ID:\t\t0x808A7\r\n\r\nLogon Type:\t\t\t2\r\n\r\nAccount For Which Logon Failed:\r\n\tSecurity ID:\t\tS-1-0-0\r\n\tAccount Name:\t\tSOC-Test\r\n\tAccount Domain:\t\tWin11-Client-01\r\n\r\n\nFailure Information:\r\n\tFailure Reason:\t\tUnknown user name or bad password.\r\n\tStatus:\t\t\t0xC000006D\r\n\tSub Status:\t\t0xC000006A\r\n\r\n\nProcess Information:\r\n\tCaller Process ID:\t0x1b6c\r\n\tCaller Process Nam
t rule.description	Multiple Windows Logon Failures
# rule.firedtimes	1
# rule.frequency	8
t rule.gdpr	IV_35.7.d, IV_32.2
t rule.groups	windows, windows_security, authentication_failures
t rule.hipaa	164.312.b
t rule.id	60204
# rule.level	10
rule.mail	false
t rule.mitre.id	T1110
t rule.mitre.tactic	Credential Access
t rule.mitre.technique	Brute Force
t rule.nist_800_53	AU.14, AC.7, SI.4
t rule.pci_dss	10.2.4, 10.2.5, 11.4
t rule.tsc	CC6.1, CC6.8, CC7.2, CC7.3
timestamp	Oct 30, 2026 @ 17:59:42.225

Analysis:
The individual alerts confirmed that failed authentication events were reaching Wazuh. Rule 60204 confirmed that Wazuh also correlated repeated failures and generated a higher-severity alert.

Earlier testing produced individual failures without the correlation alert. Inspection of the installed rule showed a frequency setting of 8, a time window of 240 seconds, and a requirement for matching source IP addresses. The earlier screenshot contained only six failures close together, which was consistent with not reaching the configured threshold.

During the later test, the dashboard showed the repeated-failure alert followed by an account-lockout alert. Windows enforced the lockout policy, Wazuh detected and reported the activity.

Scope and Limitations:
This was an authorized local repeated login failure simulation using a dedicated account inside my VM. It did not demonstrate a remote network attack or successful password compromise.

The authentication evidence came from the Windows Security log. Although Sysmon was installed, this exercise did not depend on Sysmon events.

Conclusion:
The test successfully demonstrated individual failed login detection, correlation of repeated failures, and visibility into account lockout. It also showed why the timing and grouping of events matter when validating a detection rule.

Step 14: Investigation 02 - Controlled Account Discovery and Sysmon Investigation
Investigation 02: Local Account Discovery

Date: October 30, 2026
Endpoint: Win11-Client-01
Wazuh agent ID: 002
Test account: Win11-Client-01\socadmin

Objective:
Verify that Sysmon could record local account-discovery activity and that Wazuh could collect the event and generate the alert. I also practiced reviewing the command, user, parent process, and timestamp to connect an alert to a known action.

Environment and Preparation:
I performed this test inside my Windows 11 VirtualBox VM. Sysmon was already installed, and the Wazuh agent was configured to collect events from:

- Microsoft-Windows-Sysmon/Operational

Inside the VM, I checked both services using the PowerShell command:

- Get-Service Sysmon*, WazuhSvc

Both Sysmon64 and WazuhSvc displayed as Running.

Test-Procedure:
First, I recorded the VM's current time:

- Get-Date

The output showed:

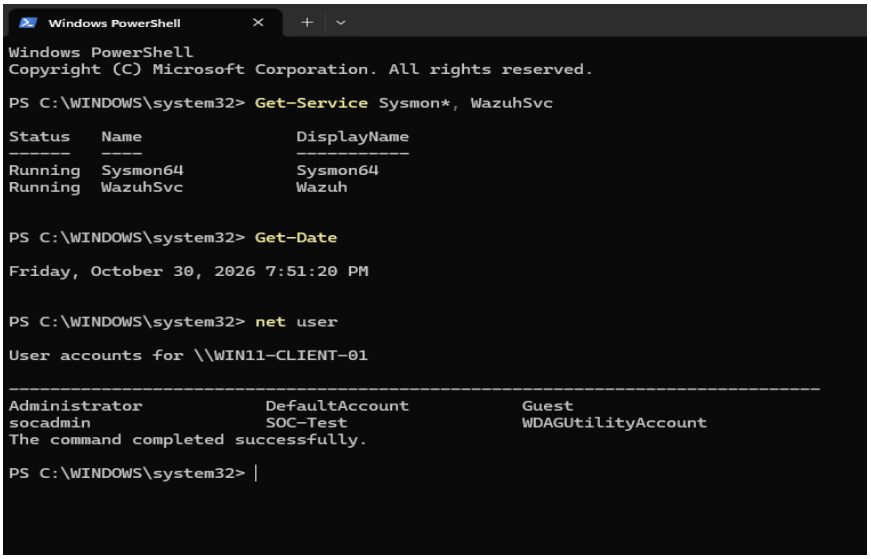
- Friday, October 30, 2026 7:51:20 PM

Then I ran:

- net user

This listed the local user accounts on Win11-Client-01, including socadmin and SOC-Test. The command completed successfully.

Running net user without additional options lists accounts. It does not change accounts or display their passwords.



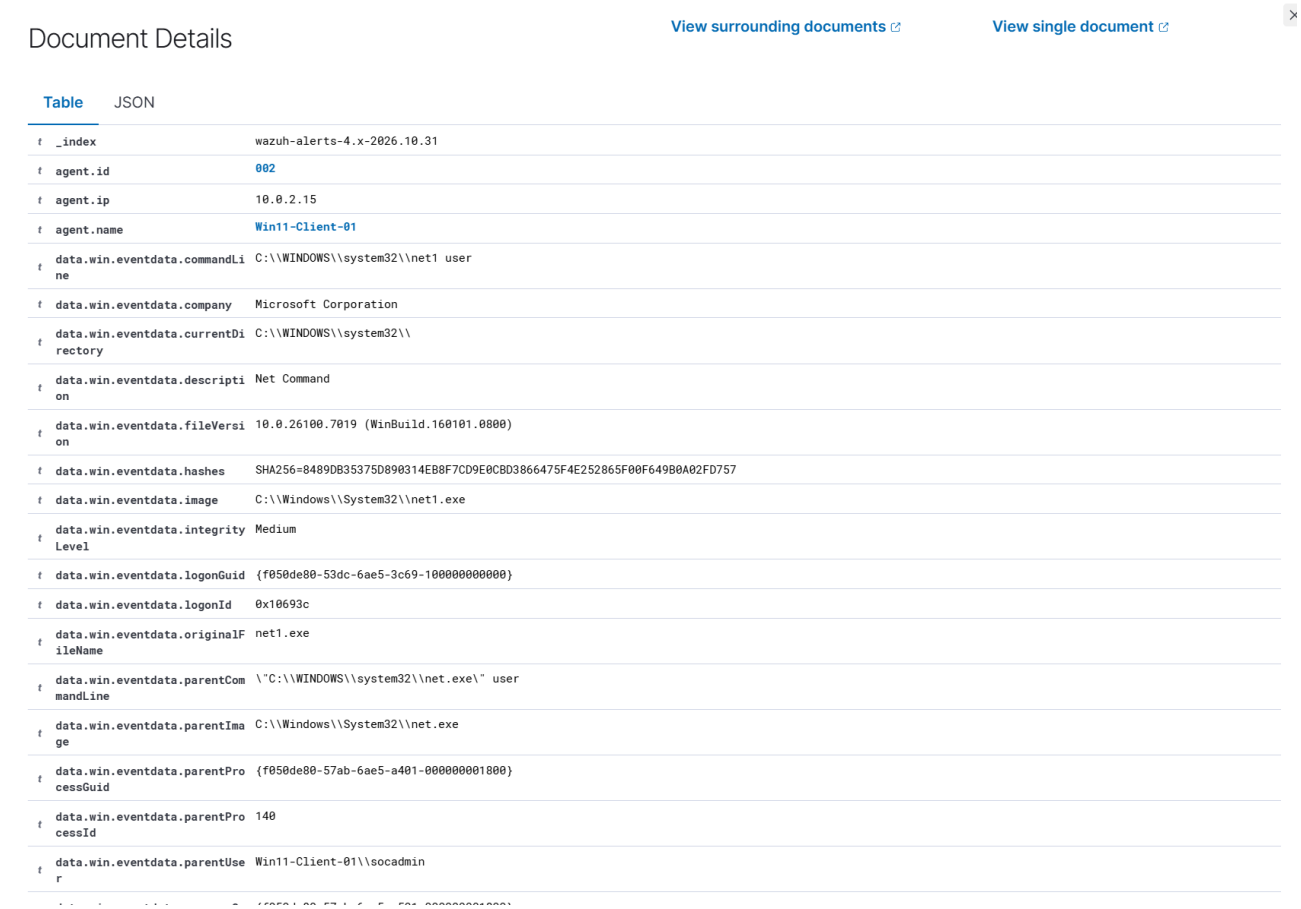
Finding the event in Wazuh:

In the Wazuh dashboard, I opened Threat Hunting → Events, selected agent 002, and set the time range to the last 15 minutes. I opened the discovery alert near my test time and reviewed its Document Details.

Results:

The event contained the following information -

Field	Observed Value
Endpoint	Win11-Client-01
Agent ID	002
User	Win11-Client-01\socadmin
Wazuh rule ID	92031
Rule description	Discovery activity executed
Rule level	3

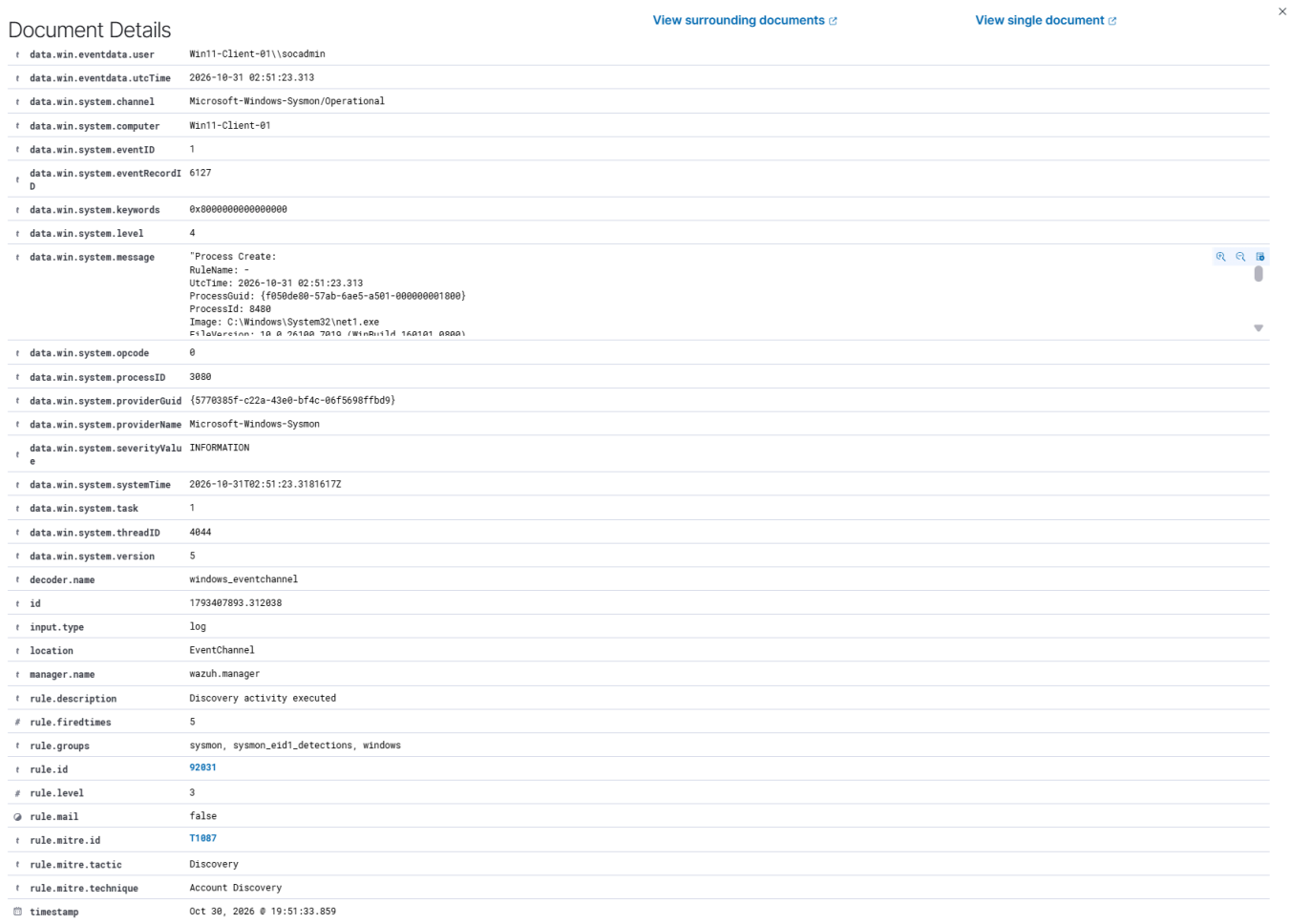


Document Details

View surrounding documents View single document

Table JSON

_index	wazuh-alerts-4.x-2026.10.31
agent.id	002
agent.ip	10.0.2.15
agent.name	Win11-Client-01
data.win.eventdata.commandLine	C:\\WINDOWS\\system32\\net1 user
data.win.eventdata.company	Microsoft Corporation
data.win.eventdata.currentDirectory	C:\\WINDOWS\\system32\\
data.win.eventdata.description	Net Command
data.win.eventdata.fileVersion	10.0.26100.7019 (WinBuild.160101.0800)
data.win.eventdata.hashes	SHA256=8489DB35375D890314EB8F7CD9E0CBD3866475F4E252865F00F649B0A02FD757
data.win.eventdata.image	C:\\Windows\\System32\\net1.exe
data.win.eventdata.integrityLevel	Medium
data.win.eventdata.logonGuid	{f050de80-53dc-6ae5-3c69-100000000000}
data.win.eventdata.logonId	0x10693c
data.win.eventdata.originalFileName	net1.exe
data.win.eventdata.parentCommandLine	\"C:\\WINDOWS\\system32\\net.exe\" user
data.win.eventdata.parentImage	C:\\Windows\\System32\\net.exe
data.win.eventdata.parentProcessGuid	{f050de80-57ab-6ae5-a401-000000001800}
data.win.eventdata.parentProcessId	140
data.win.eventdata.parentUser	Win11-Client-01\\socadmin



Document Details

View surrounding documents | View single document

data.win.eventdata.user	Win11-Client-01\\socadmin
data.win.eventdata.utcTime	2026-10-31 02:51:23.313
data.win.system.channel	Microsoft-Windows-Sysmon/Operational
data.win.system.computer	Win11-Client-01
data.win.system.eventID	1
data.win.system.eventRecordID	6127
data.win.system.keywords	0x8000000000000000
data.win.system.level	4
data.win.system.message	"Process Create: RuleName: - UtcTime: 2026-10-31 02:51:23.313 ProcessGuid: {f050de80-57ab-6ae5-a501-000000001800} ProcessId: 8480 Image: C:\Windows\System32\net1.exe FileVersion: 10.0.26100.7019 (WinBuild.160101.0800)
data.win.system.opcode	0
data.win.system.processID	3080
data.win.system.providerGuid	{5770385f-c22a-43e0-bf4c-06f5698ffbd9}
data.win.system.providerName	Microsoft-Windows-Sysmon
data.win.system.severityValue	INFORMATION
data.win.system.systemTime	2026-10-31T02:51:23.3181617Z
data.win.system.task	1
data.win.system.threadID	4044
data.win.system.version	5
decoder.name	windows_eventchannel
id	1793407893.312038
input.type	log
location	EventChannel
manager.name	wazuh.manager
rule.description	Discovery activity executed
rule.firedtimes	5
rule.groups	sysmon, sysmon_eid1_detections, windows
rule.id	92031
rule.level	3
rule.mail	false
rule.mitre.id	T1087
rule.mitre.tactic	Discovery
rule.mitre.technique	Account Discovery
timestamp	Oct 30, 2026 @ 19:51:33.859

Analysis:

The recorded net1.exe process had net.exe user as its parent command. This connected the event to the account-listing command I ran.

I compared the event with my test using four details:

- The endpoint was Win11-Client-01.
- The user was Win11-Client-01\socadmin.
- The recorded command matched the account-discovery activity.
- The timestamp was close to the time I recorded before running the command.

The Wazuh dashboard timestamp was approximately 13.9 seconds after the Get-Date output. This interval includes the time between entering the commands and any event collection and processing delay. It is not a precise measurement of detection speed.

Wazuh classified the activity as account discovery and generated a level 3 alert. In this case, the action was an authorized lab test. The same command could also be used during legitimate administration or by an attacker gathering information about available accounts.

This demonstrated why an alert requires context. The command alone does not prove malicious intent; the user, parent process, timing, and surrounding activity help determine what happened.

Scope and limitations:

This exercise tested local account discovery and process monitoring. It did not involve credential dumping, password theft, privilege escalation, or a remote attack.

Unlike the previous failed login investigation, which used the Windows Security log, this investigation used Sysmon process creation telemetry.

The result confirms detection of this specific command in my current configuration. It does not prove that every account-discovery method would generate an alert.

Conclusion:

The test successfully demonstrated that Sysmon recorded account-discovery activity, the Wazuh agent forwarded the event, and Wazuh generated rule 92031 — Discovery activity executed.

I verified the alert by matching its endpoint, user, command, parent process, and timestamp with my manual test. This completed my second investigation and demonstrated process level monitoring alongside the authentication monitoring tested in Investigation 01.